

Name: Nguyễn Đăng Phương Duy

ID:23520372

Class: IT007.P28.2

OPERATING SYSTEM LAB 5'S REPORT

SUMMARY

Task		Status	Page
Section 5.5	Ex 1	Hoàn thành	2
	Ex 2	Hoàn thành	4
	Ex 3	Hoàn thành	7
	Ex 4	Hoàn thành	8

Self-scores: 10

Note: Export file to **PDF and name the file by following format:
Student ID_LABx.pdf*

Section 4.5

1. Bài tập 1

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define MAX_BUFFER 372 // MSSV 23520372

int products = 0;
int sells = 0;

sem_t sem_product;
sem_t sem_store;
```

Đầu tiên, ta tiến hành khai báo thư viện, cùng với các biến toàn cục và MAX_BUFFER là 4 số cuối MSSV (0372) để làm điều kiện cho bài toán. Khởi tạo 2 semaphore để kiểm tra điều kiện cho bài toán với sem_product để thông báo là có hàng và sem_store để thông báo còn chỗ trống để chứa hàng mới.

```
void* producer(void* arg){
    while (1){
        sem_wait(&sem_store); // chờ bán nếu kho đầy (products == sells + 371)

        products++;
        printf("[Producer] Product: %d, Sell: %d\n", products, sells);

        sem_post(&sem_product); // báo cho seller là có hàng
        sleep(0);
    }
    return NULL;
}

void* seller(void* arg){
    while (1){
        sem_wait(&sem_product); // chờ nếu chưa có hàng (products == sells)

        sells++;
        printf("[Seller] Product: %d, Sell: %d\n", products, sells);
    }
}
```

```

        sem_post(&sem_store); // báo cho producer là có thêm chỗ trống
        sleep(1);
    }
    return NULL;
}

```

Kế tiếp ta tạo hai function là producer và seller để xử lý các thao tác sản xuất hàng và bán hàng

- Producer: đầu tiên sẽ kiểm tra xem trong kho còn chỗ trống hay không bằng cách sử dụng semaphore sem_store. Nếu còn chỗ trống sẽ tiến hành sản xuất mặt hàng và tăng biến products lên 1, nếu không còn chỗ trống sẽ chờ bán hàng. Cuối cùng sẽ thông báo cho semaphore sem_product rằng đã sản xuất thêm 1 mặt hàng.
- Seller: đầu tiên sẽ kiểm tra xem có còn hàng trong kho để bán hay không bằng cách sử dụng semaphore sem_product. Nếu còn sẽ tiến hành bán hàng và tăng sells lên 1, nếu không có hàng sẽ tiến hành đợi producer sản xuất thêm hàng. Cuối cùng sẽ thông báo cho semaphore sem_store rằng kho hàng đã bán đi 1 mặt hàng.

```

int main(){
    pthread_t t1, t2;

    // Khởi tạo semaphore
    sem_init(&sem_product, 0, 0); // Ban đầu chưa có sản phẩm
    sem_init(&sem_store, 0, MAX_BUFFER); // Tối đa 372 sản phẩm (products
- sells <= 371)

    pthread_create(&t1, NULL, producer, NULL);
    pthread_create(&t2, NULL, seller, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem_product);
    sem_destroy(&sem_store);

    return 0;
}

```

Cuối cùng tại hàm main, ta khởi tạo các semaphore và các tiến trình để tiến hành mô phỏng bài toán.

* Kết quả sau khi chạy code :

```

✖ nguyendangphuongduy-23520372@ZUYYYY:~/lab5$ ./bai1.exe
[Producer] Product: 1, Sell: 0
[Producer] Product: 2, Sell: 0
[Seller] Product: 2, Sell: 1
[Producer] Product: 3, Sell: 1
[Producer] Product: 4, Sell: 1
[Producer] Product: 5, Sell: 1
[Seller] Product: 5, Sell: 2
[Producer] Product: 6, Sell: 2
[Seller] Product: 6, Sell: 3
[Producer] Product: 7, Sell: 3
[Seller] Product: 7, Sell: 4
[Producer] Product: 8, Sell: 4
[Seller] Product: 8, Sell: 5

```

Hình 1: Kết quả chạy chương trình

2. Bài tập 2

```

int main(){
    srand(time(NULL));
    printf("Nhap n: ");
    scanf("%d", &n);
    a = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++){
        a[i] = -1;
    }

    sem_init(&sem_create, 0, n);
    sem_init(&sem_consume, 0, 0);
    pthread_t writer, reader;

    pthread_create(&writer, NULL, create, NULL);
    pthread_create(&reader, NULL, consume, NULL);

    pthread_join(writer, NULL);
    pthread_join(reader, NULL);

    return 0;
}

```

Tại hàm main, ta cấp phát động mảng a có n phần tử với n được nhập từ bàn phím, các phần tử của mảng a có giá trị mặc định là -1 khi chưa nhập giá trị.

Ta cũng khởi tạo 2 semaphore phục vụ cho việc đồng bộ tiểu trình, trong đó sem_create để kiểm tra mảng còn phần tử trống để thêm vào hay không và sem_consume để kiểm tra mảng còn phần tử nào để lấy ra hay không.

Tiếp theo ta tiến hành gọi các tiểu trình thêm phần tử vào mảng và tiểu trình lấy phần tử ra khỏi mảng.

Trước tiên, xét trường hợp tiểu trình không được đồng bộ bởi semaphore:

```
void* create(void* arg){
    while (1){
        //sem_wait(&sem_consume);
        int size = 0;
        while(size < n && a[size] != -1){
            size++;
        }
        int num = rand() % 100;
        a[size] = num;
        printf("[Create] Create: %d. Array size: %d\n", num, size + 1);
        //sem_post(&sem_create);
    }
    return NULL;
}

void* consume(void* arg){
    while (1){
        //sem_wait(&sem_create);
        int size = 0;
        while(size < n && a[size] != -1){
            size++;
        }
        if (size > 0){
            int index = size % 2;
            int value = a[index];
            a[index] = a[size - 1];
            a[size - 1] = -1;
            size--;
            printf("[Consume] Removed %d from array. Array size = %d\n", value,
size);
        }
        else{
            printf("[Consume] Nothing in array a\n");
        }
        //sem_post(&sem_consume);
    }
    return NULL;
}
```

Khi ta tiến hành chạy chương trình sẽ phát sinh ra lỗi:

```
[Consume] Nothing in array a
[Create] Create: 17. Array size: 11
[Create] Create: 28. Array size: 1
[Create] Create: 3. Array size: 3
[Create] Create: 19. Array size: 4
[Create] Create: 35. Array size: 5
[Create] Create: 29. Array size: 6
[Create] Create: 97. Array size: 7
[Create] Create: 41. Array size: 8
[Create] Create: 77. Array size: 9
[Create] Create: 44. Array size: 10
[Create] Create: 24. Array size: 11
[Create] Create: 66. Array size: 11
[Create] Create: 6. Array size: 11
[Consume] Nothing in array a
[Consume] Removed 28 from array. Array size = 9
[Consume] Removed 83 from array. Array size = 8
[Consume] Removed 44 from array. Array size = 7
[Consume] Removed 77 from array. Array size = 6
[Consume] Removed 41 from array. Array size = 5
[Create] Create: 87. Array size: 11
[Create] Create: 79. Array size: 5
[Create] Create: 5. Array size: 6
```

Hình 2: Lỗi khi không đồng bộ bằng semaphore

Có thể thấy khi không được đồng bộ bởi semaphore, các tiểu trình create và consume sẽ chạy không kiểm soát được khiến việc lấy giá trị và ghi thêm giá trị vào mảng bị lỗi, và đôi lúc giá trị được ghi vào mảng vượt quá số lượng phần tử của mảng.

Khi đó chúng ta thêm vào các semaphore để tiến hành đồng bộ 2 tiểu trình lại:

```

[Create] Create: 76. Array size: 2
[Create] Create: 1. Array size: 3
[Create] Create: 49. Array size: 4
[Create] Create: 79. Array size: 5
[Create] Create: 76. Array size: 6
[Create] Create: 5. Array size: 7
[Create] Create: 20. Array size: 8
[Create] Create: 72. Array size: 8
[Create] Create: 75. Array size: 9
[Consume] Removed 55 from array. Array size = 7
[Consume] Removed 76 from array. Array size = 8
[Consume] Removed 20 from array. Array size = 7
[Consume] Removed 75 from array. Array size = 6
[Consume] Removed 72 from array. Array size = 5
[Consume] Removed 5 from array. Array size = 4
[Consume] Removed 76 from array. Array size = 3
[Consume] Removed 79 from array. Array size = 2
[Consume] Removed 49 from array. Array size = 1
^C

```

Hình 3: Kết quả sau khi đồng bộ tiểu trình bằng semaphore

3. Bài tập 3

```

int main(){
    pthread_t PA, PB;

    pthread_create(&PA, NULL, processA, NULL);
    pthread_create(&PB, NULL, processB, NULL);

    pthread_exit(NULL);
    return 0;
}

```

Đầu tiên tại hàm main, ta khởi tạo hai tiểu trình và tiến hành chạy song song hai tiểu trình này.

```

void* processA(void* arg){
    while (1){
        int temp = x;
        usleep(123456);
        x = temp + 1;
        if (x == 20)
            x = 0;
    }
}

```

```

        printf("A: x = %d\n", x);
        usleep(123456);
    }
    return NULL;
}

void* processB(void* arg){
    while (1){
        int temp = x;
        usleep(123456);
        x = temp + 1;
        if (x == 20)
            x = 0;
        printf("B: x = %d\n", x);
    }
    return NULL;
}

```

Hai hàm được thể hiện như đề, ngoài ra có thêm các hàm `usleep()` nhằm làm chậm quá trình thực thi của hai hàm để dễ dàng quan sát và bắt gặp lỗi.

Sau khi chương trình được thực thi, ta bắt gặp lỗi là các biến `x` được in ra có thể bị trùng nhau => xảy ra race condition.

```

❗ nguyendangphuongduy-23520372@ZUYYY:~/lab5$ ./bai3.exe
B: x = 1
A: x = 1
B: x = 2
A: x = 2
B: x = 3
B: x = 4
A: x = 4
B: x = 5
B: x = 6
A: x = 6
B: x = 7

```

Hình 4: Lỗi race condition

4. Bài tập 4

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int x = 0;
pthread_mutex_t lock;

```



```

void* processA(void* arg){
    while (1){
        pthread_mutex_lock(&lock);

        int temp = x;
        usleep(123456);
        x = temp + 1;
        if (x == 20)
            x = 0;
        printf("A: x = %d\n", x);

        pthread_mutex_unlock(&lock);
        usleep(123456);
    }
    return NULL;
}

void* processB(void* arg){
    while (1){
        pthread_mutex_lock(&lock);

        int temp = x;
        usleep(123456);
        x = temp + 1;
        if (x == 20)
            x = 0;
        printf("B: x = %d\n", x);

        pthread_mutex_unlock(&lock);
        usleep(123456);
    }
    return NULL;
}

int main(){
    pthread_t PA, PB;
    pthread_mutex_init(&lock, NULL);

    pthread_create(&PA, NULL, processA, NULL);
    pthread_create(&PB, NULL, processB, NULL);

    pthread_exit(NULL);
    return 0;
}

```

Để giải quyết vấn đề bất hợp lý tại bài 3, ta tiến hành thêm phương pháp đồng bộ với mutex, nhằm đảm bảo biến x chỉ được thực hiện bởi một trong hai tiểu trình A hoặc B. Đoạn code sửa lỗi cho bài 3 như trên. Sau đó tiến hành chạy chương trình và từ đó thu được kết quả không xảy ra hiện tượng trùng lặp giá trị của biến x như hình dưới:

```
ⓧ nguyendangphuongduy-23520372@ZUYYYY:~/lab5$ ./bai4.exe
A: x = 1
B: x = 2
A: x = 3
B: x = 4
A: x = 5
B: x = 6
A: x = 7
B: x = 8
A: x = 9
B: x = 10
A: x = 11
B: x = 12
A: x = 13
B: x = 14
A: x = 15
```

Hình 5: Kết quả sau khi thêm mutex